



I. DATOS GENERALES

Tema:	Búsquedas Informadas aplicadas al 8-Puzzle
Unidad de Organización Curricular:	PROFESIONAL
Nivel y Paralelo:	7mo semestre
Alumnos participantes:	López Ocaña Jonathan Josue Sanchez Guevara Christian Alejandro
Asignatura:	Inteligencia Artificial
Docente:	Ing. Ruben Nogales, Mg.

1. Introducción

Este documento analiza dos algoritmos de búsqueda informada aplicados al problema del 8-Puzzle: *Primero el Mejor* (*Greedy Best-First Search*) y A^* . Ambos parten del mismo estado inicial, utilizan la misma heurística y trabajan sobre el mismo espacio de estados.

La única diferencia entre ellos es la función empleada para decidir qué nodo explorar a continuación. Esta diferencia en el criterio de selección es responsable de las principales diferencias observadas en los resultados experimentales, tanto en la longitud de las soluciones como en el número de nodos explorados.

2. Planteamiento del problema

El 8-Puzzle es un problema clásico de búsqueda en Inteligencia Artificial que consiste en un tablero de 3×3 compuesto por ocho fichas numeradas y un espacio vacío. El objetivo es encontrar una secuencia de movimientos que permita transformar un estado inicial determinado en un estado objetivo.

Debido a que el número de configuraciones posibles del tablero es elevado, una búsqueda no informada puede explorar una gran cantidad de estados antes de encontrar una solución. Por esta razón, resulta necesario utilizar estrategias de búsqueda informada que incorporen conocimiento adicional sobre la distancia entre un estado y el objetivo.

En este trabajo se plantea el problema de comparar el comportamiento de dos algoritmos de búsqueda informada: *Primero el Mejor* (*Greedy Best-First Search*) y A^* . Ambos algoritmos utilizan la distancia Manhattan como heurística y trabajan sobre el mismo espacio de estados, permitiendo analizar cómo el criterio utilizado para seleccionar el siguiente



nodo afecta el proceso de búsqueda.

El problema central consiste, por tanto, en determinar cuál de los dos algoritmos presenta un mejor comportamiento para resolver una instancia del 8-Puzzle, considerando principalmente la cantidad de movimientos necesarios para alcanzar la solución, el número de nodos expandidos y el tiempo de ejecución.

Para el experimento se parte del siguiente estado inicial:

7	2	4
5	—	6
8	3	1

y se busca alcanzar el estado objetivo:

1	2	3
4	5	6
7	8	—

A partir de esta instancia se ejecutan ambos algoritmos bajo las mismas condiciones, utilizando la distancia Manhattan como función heurística. Los resultados obtenidos permiten establecer las ventajas y limitaciones de cada estrategia y determinar cuál resulta más conveniente cuando se requiere una solución óptima.

La comparación es especialmente relevante debido a que, aunque ambos algoritmos utilizan la misma información heurística, la emplean de manera diferente. Primero el Mejor considera únicamente la estimación del costo restante,

$$f(n) = h(n),$$

mientras que A* considera tanto el costo acumulado como la estimación restante,

$$f(n) = g(n) + h(n).$$

Esta diferencia constituye el elemento principal que se pretende analizar experimentalmente.

3. Solubilidad y espacio de estados

No todas las configuraciones posibles del 8-Puzzle pueden alcanzarse desde un estado inicial determinado. Aunque el tablero contiene nueve posiciones, incluyendo el espacio vacío, existen



$$9! = 362\,880$$

configuraciones diferentes al considerar todas las posibles permutaciones de las ocho fichas y el espacio vacío.

Sin embargo, estas configuraciones se dividen en dos clases de paridad, de las cuales únicamente una es alcanzable desde un estado inicial determinado. Por esta razón, para cualquier configuración inicial existen solamente

$$\frac{9!}{2} = 181\,440$$

estados alcanzables.

Control de solubilidad

Antes de ejecutar los algoritmos de búsqueda se implementó un control de solubilidad que determina si el estado inicial puede alcanzar el estado objetivo. Este control evita ejecutar innecesariamente los algoritmos sobre una instancia que no posee solución.

Para ello se utiliza la propiedad de las **inversiones**. Una inversión se produce cuando una ficha de menor valor aparece después de una ficha de mayor valor al recorrer el tablero de izquierda a derecha y de arriba hacia abajo, ignorando el espacio vacío.

Para un tablero de 3×3 , el problema es soluble cuando la paridad del número de inversiones del estado inicial coincide con la del estado objetivo. En consecuencia, antes de iniciar la búsqueda se verifica esta condición.

Este control permite garantizar que la instancia proporcionada a los algoritmos pertenece al conjunto de estados alcanzables y evita realizar una búsqueda sobre una configuración que no puede llegar al estado objetivo.

4. Ejecución del programa y almacenamiento de resultados

La ejecución del programa se realiza desde la terminal mediante el comando:

```
python main.py
```

El archivo `main.py` se encarga de iniciar la ejecución de los algoritmos y generar los resultados correspondientes a la instancia del 8-Puzzle.

Los resultados de cada algoritmo se almacenan automáticamente en archivos de texto (`.txt`), utilizando nombres que permiten identificar directamente el algoritmo empleado. Los archivos generados son:



- resultados_primeros_mejor.txt: resultados obtenidos mediante Primeros el Mejor.
- resultados_a_estrella.txt: resultados obtenidos mediante A*.

Estos archivos contienen información como el número de movimientos de la solución, los nodos expandidos, el tiempo de ejecución y el camino seguido para alcanzar el estado objetivo.

5. Heurística utilizada

Para resolver el problema del 8-Puzzle se utiliza la **distancia Manhattan** como función heurística. Para cada ficha numerada del 1 al 8, sin considerar el espacio vacío, se calcula la distancia entre su posición actual y la posición que debe ocupar en el estado meta.

La distancia Manhattan de una ficha i se define como:

$$d_i = |r_i - r_i^*| + |c_i - c_i^*|.$$

donde (r_i, c_i) representa la posición actual de la ficha i y (r_i^*, c_i^*) representa su posición en el estado objetivo.

La función heurística utilizada por ambos algoritmos corresponde a la suma de las distancias Manhattan de todas las fichas:

$$h(n) = \sum_{i=1}^8 (|r_i - r_i^*| + |c_i - c_i^*|).$$

Comparación con otras heurísticas

Existen diferentes heurísticas que pueden utilizarse para el 8-Puzzle. Las principales son:

Cuadro 1: Comparación de heurísticas para el 8-Puzzle

Heurística	Admisible	Información	Ventaja principal
Fichas mal colocadas	Sí	Baja	Solo cuenta fichas fuera de lugar.
Distancia Manhattan	Sí	Alta	Estima la distancia de cada ficha al objetivo.
Conflictos lineales	Sí	Muy alta	Considera conflictos entre fichas.
Heurística 0	Sí	Nula	No aporta información sobre la meta.

La heurística de fichas mal colocadas únicamente cuenta cuántas fichas no se encuentran en su posición correcta. Por ejemplo, una ficha que está a una posición de su objetivo y



otra que está a tres posiciones contribuyen de la misma manera. En cambio, la distancia Manhattan distingue ambas situaciones, proporcionando una estimación más cercana al costo restante real.

En términos generales, mientras más cercana sea la estimación $h(n)$ al costo real restante $h^*(n)$, mayor información proporciona la heurística sobre el estado actual. La distancia Manhattan aprovecha la estructura del problema para obtener esta información, ya que considera cuántos desplazamientos horizontales y verticales necesita cada ficha.

Por esta razón, Manhattan ofrece un buen equilibrio entre **información y costo computacional**. Es más informativa que contar fichas mal colocadas, pero sigue siendo sencilla y rápida de calcular. Aunque los conflictos lineales pueden proporcionar una estimación aún más precisa, requieren un cálculo adicional que no resulta necesario para este proyecto.

Justificación de la heurística

La distancia Manhattan es adecuada para el 8-Puzzle porque cada movimiento válido desplaza una ficha una posición horizontal o verticalmente. Por lo tanto, representa el número mínimo de desplazamientos que cada ficha necesita para alcanzar su posición objetivo, ignorando temporalmente las restricciones producidas por las demás fichas.

Las restricciones del tablero pueden obligar a realizar movimientos adicionales, pero nunca pueden reducir este mínimo. Por ello, la distancia Manhattan proporciona una estimación útil y cercana al costo restante del problema.

Esta característica es especialmente importante para A^* , ya que una heurística más informativa permite ordenar mejor los estados de la frontera y reducir la cantidad de nodos explorados.

Admisibilidad y consistencia

Para que A^* pueda garantizar que la solución encontrada es óptima, la heurística debe ser **admisibile**. Una heurística $h(n)$ es admisible si nunca sobreestima el costo real mínimo necesario para alcanzar el estado objetivo:

$$h(n) \leq h^*(n),$$

donde $h^*(n)$ representa el costo real mínimo desde el estado n hasta el estado objetivo.

La distancia Manhattan cumple esta propiedad porque cada ficha debe desplazarse, como mínimo, el número de filas y columnas indicado por su distancia Manhattan. Por tanto:

$$h(n) \leq h^*(n).$$



Además, la distancia Manhattan es **consistente** o monótona. Para un nodo n y un sucesor n' se cumple:

$$h(n) \leq c(n, n') + h(n').$$

Como cada movimiento tiene costo unitario:

$$c(n, n') = 1.$$

Por lo tanto:

$$h(n) \leq 1 + h(n').$$

La consistencia garantiza que los valores de la función de evaluación de A^* ,

$$f(n) = g(n) + h(n),$$

no disminuyan a lo largo de un camino, permitiendo mantener la garantía de optimalidad al utilizar estados visitados.

En conclusión, la distancia Manhattan se selecciona porque es **admisible, consistente, informativa y de bajo costo computacional**. Además, proporciona una estimación del costo restante más precisa que heurísticas simples como las fichas mal colocadas, sin introducir una complejidad de cálculo significativa. Por estas razones resulta adecuada para comparar el comportamiento de Primero el Mejor y A^* .

6. Los dos métodos de búsqueda

6.1. Primero el Mejor (*Greedy Best-First Search*)

El algoritmo Primero el Mejor utiliza únicamente la función heurística para determinar qué nodo debe explorarse:

$$f(n) = h(n).$$

En cada iteración, se selecciona de la frontera el nodo cuyo estado presenta el menor valor de $h(n)$. La frontera se implementa mediante una **cola de prioridad**, de manera que el nodo con menor valor heurístico tenga mayor prioridad para ser explorado.

Para evitar procesar repetidamente los mismos estados, se utiliza una estructura de datos destinada a almacenar los estados ya visitados. En la implementación realizada, esta



estructura permite comprobar si un suceso ya fue explorado antes de incorporarlo nuevamente a la frontera. De esta manera, se evita generar ciclos innecesarios y se reduce el número de estados procesados.

El procedimiento general puede resumirse de la siguiente manera:

1. Se coloca el estado inicial en la cola de prioridad.
2. Se extrae el nodo con menor valor de $h(n)$.
3. Se comprueba si el estado corresponde al estado objetivo.
4. Si no se ha alcanzado la solución, el estado se marca como visitado.
5. Se generan sus estados sucesores.
6. Los sucesores que no han sido visitados se incorporan a la frontera.
7. El proceso se repite hasta encontrar el estado objetivo o hasta que la frontera quede vacía.

Es un algoritmo “codicioso”, ya que persigue el estado que parece mejor en el momento actual, incluso si para alcanzarlo debe recorrer un camino largo. Esto significa que no considera el costo acumulado $g(n)$ del camino recorrido, sino únicamente la estimación proporcionada por la heurística.

Por esta razón, Primero el Mejor no garantiza obtener una solución óptima. En espacios infinitos tampoco garantiza completitud, aunque en el caso del 8-Puzzle el espacio de estados es finito.

En el peor caso, su complejidad temporal y espacial puede expresarse como:

$$O(b^m),$$

donde b es el factor de ramificación y m representa la máxima profundidad alcanzada durante la búsqueda, que no necesariamente coincide con la profundidad de la solución encontrada.

Resultado obtenido

Para la instancia utilizada en este trabajo, Primero el Mejor encontró una solución de **62 movimientos**. El resultado de la ejecución se muestra en la Figura 1.


```
Paso 1
Movimiento: ABAJO
7 2 4
5 3 6
8 _ 1
g(n) = 1
h(n) = 13
f(n) = 14

Paso 2
Movimiento: DERECHA
7 2 4
5 3 6
8 1 _
g(n) = 2
h(n) = 12
f(n) = 14

Paso 3
Movimiento: ARRIBA
7 2 4
5 3
```

Figura 1: Resultado de la ejecución del algoritmo Primero el Mejor.

Secuencia de movimientos

La solución encontrada por el algoritmo Primero el Mejor consta de **62 movimientos**.
La secuencia obtenida es:

ABAJO → DERECHA → ARRIBA → IZQUIERDA → IZQUIERDA → ARRIBA → DERECHA
→ DERECHA → ABAJO → ABAJO → IZQUIERDA → IZQUIERDA → ARRIBA →
DERECHA → ARRIBA → IZQUIERDA → ABAJO → DERECHA → ABAJO → DERECHA →
ARRIBA → IZQUIERDA → ABAJO → IZQUIERDA → ARRIBA → DERECHA → DERECHA
→ ABAJO → IZQUIERDA → ARRIBA → ARRIBA → IZQUIERDA → ABAJO → ABAJO
→ DERECHA → DERECHA → ARRIBA → IZQUIERDA → ABAJO → IZQUIERDA →
ARRIBA → DERECHA → DERECHA → ABAJO → IZQUIERDA → IZQUIERDA → ARRIBA
→ ARRIBA → DERECHA → ABAJO → ABAJO → DERECHA → ARRIBA → ARRIBA →
IZQUIERDA → IZQUIERDA → ABAJO → DERECHA → ARRIBA → DERECHA → ABAJO
→ ABAJO

La secuencia comienza en el estado inicial y finaliza en el estado objetivo, por lo que corresponde a una solución de longitud 62.

6.2. A*

El algoritmo A* combina el costo real acumulado desde el estado inicial con el costo heurístico estimado hasta el estado objetivo. Para determinar qué nodo debe explorarse



a continuación utiliza la función de evaluación:

$$f(n) = g(n) + h(n).$$

Aquí, $g(n)$ representa el costo real del camino recorrido desde el estado inicial hasta el nodo n , mientras que $h(n)$ representa el costo estimado desde n hasta el estado objetivo. En el problema del 8-Puzzle, cada movimiento tiene un costo unitario, por lo que $g(n)$ corresponde directamente al número de movimientos realizados para llegar al estado actual.

La utilización conjunta de ambos valores permite que A^* considere tanto el camino recorrido como la distancia estimada que falta para alcanzar la solución. De esta manera, evita concentrarse únicamente en estados que parecen cercanos al objetivo pero que han sido alcanzados mediante caminos innecesariamente largos.

La frontera del algoritmo se implementa mediante una **cola de prioridad**, en la cual los nodos se ordenan de acuerdo con el menor valor de $f(n)$. Por tanto, en cada iteración se selecciona para su expansión el nodo con el menor valor de:

$$f(n) = g(n) + h(n).$$

Al igual que en Primero el Mejor, se utiliza una estructura de datos para almacenar los estados que ya han sido visitados. Antes de agregar un sucesor a la frontera, se comprueba si dicho estado ya fue explorado, evitando así procesar repetidamente configuraciones previamente visitadas y reduciendo la aparición de ciclos durante la búsqueda.

El procedimiento utilizado puede resumirse mediante los siguientes pasos:

1. Se coloca el estado inicial en la cola de prioridad.
2. Se extrae de la frontera el nodo con menor valor de $f(n)$.
3. Se comprueba si el estado corresponde al estado objetivo.
4. Si no se ha alcanzado la solución, el nodo se marca como visitado.
5. Se generan los estados sucesores del nodo actual.
6. Para cada sucesor se calculan los valores $g(n)$, $h(n)$ y $f(n) = g(n) + h(n)$.
7. Los estados que no han sido visitados se incorporan a la frontera de acuerdo con su valor de $f(n)$.
8. El proceso continúa hasta encontrar el estado objetivo.



Garantía de optimalidad

Una de las principales ventajas de A* frente a Primero el Mejor es que puede garantizar una solución óptima cuando se utiliza una heurística admisible.

Como se explicó anteriormente, la distancia Manhattan utilizada en este trabajo cumple:

$$h(n) \leq h^*(n),$$

donde $h^*(n)$ representa el costo real mínimo desde el nodo n hasta el estado objetivo.

Esta propiedad garantiza que la heurística nunca sobreestima el costo restante. Por consiguiente, A* puede comparar el costo acumulado $g(n)$ con la estimación $h(n)$ sin descartar prematuramente un camino que pueda conducir a una solución óptima.

Además, la distancia Manhattan es consistente, por lo que satisface:

$$h(n) \leq c(n, n') + h(n')$$

para cualquier sucesor n' de n . Como cada movimiento del 8-Puzzle tiene costo unitario, se tiene:

$$h(n) \leq 1 + h(n').$$

Esta propiedad permite que los valores de $f(n)$ sean no decrecientes a lo largo de los caminos explorados y facilita el manejo de estados repetidos.

Por estas razones, cuando A* selecciona el estado objetivo con el menor valor de $f(n)$, el camino encontrado corresponde a una solución de costo mínimo.

Complejidad

En el peor caso, la complejidad temporal y espacial de A* puede expresarse como:

$$O(b^d),$$

donde b representa el factor de ramificación y d corresponde a la profundidad de la solución óptima.

La complejidad espacial también puede alcanzar:

$$O(b^d),$$

debido a que A^* debe mantener en memoria los nodos de la frontera y los estados necesarios para controlar los nodos ya visitados.

En la práctica, el número de nodos explorados depende considerablemente de la calidad de la heurística. Una heurística más informativa permite que A^* distinga mejor entre estados prometedores y estados poco prometedores, reduciendo la cantidad de nodos que necesitan ser expandidos.

En este trabajo, la utilización de la distancia Manhattan permite aprovechar una heurística admisible y consistente, obteniendo una solución óptima para la instancia analizada.

Resultado obtenido

Para la instancia utilizada en este experimento, el algoritmo A^* encontró una solución de **20 movimientos**, con un total de **747 sucesores examinados** y un tiempo de ejecución de:

$$t_{A^*} = 0.00860350 \text{ s.}$$

El resultado de la ejecución se muestra en la Figura 2.

```
=====
CAMINO DE LA SOLUCIÓN
=====

Paso 0
7 2 4
5 _ 6
8 3 1
g(n) = 0
h(n) = 14
f(n) = 14

Paso 1
Movimiento: ABAJO
7 2 4
5 3 6
8 _ 1
g(n) = 1
h(n) = 13
f(n) = 14
```

Figura 2: Resultado de la ejecución del algoritmo A^* .

Secuencia de movimientos

La solución encontrada por A^* consta de **20 movimientos**. La secuencia obtenida durante la ejecución fue:



ABAJO → DERECHA → ARRIBA → IZQUIERDA → IZQUIERDA → ARRIBA → DERECHA
→ DERECHA → ABAJO → IZQUIERDA → ABAJO → IZQUIERDA → ARRIBA →
DERECHA → ARRIBA → IZQUIERDA → ABAJO → DERECHA → DERECHA → ABAJO

La secuencia comienza en el estado inicial y termina en el estado objetivo. Dado que la distancia Manhattan es una heurística admisible y consistente, la solución de 20 movimientos obtenida por A* es óptima para la instancia analizada.

7. Resumen de resultados obtenidos

Los resultados obtenidos fueron:

Cuadro 2: Comparación de los algoritmos

Métrica	Primero el Mejor	A*
Movimientos de la solución	62	20
Nodos expandidos (contador del código)	1499	747
Tiempo de ejecución	~ 0.0165 s	~ 0.0166 s
¿Solución óptima garantizada?	No	Sí

El tiempo de ejecución no constituye un buen diferenciador en este experimento. Para instancias cuya ejecución toma apenas unos milisegundos, el ruido producido por el sistema puede ser comparable con la diferencia real entre los algoritmos.

Por ello, una métrica más confiable para comparar el comportamiento de los algoritmos es el número de nodos expandidos.

8. Nodos expandidos

Los 62 movimientos de la solución corresponden únicamente al camino final desde el estado inicial hasta la meta. Esto no representa el número de nodos explorados durante la búsqueda, ya que los algoritmos examinan múltiples ramas antes de encontrar la solución.

Además, el número de sucesores depende de la posición del espacio vacío:

- Esquina: 2 sucesores.
- Borde: 3 sucesores.
- Centro: 4 sucesores.

Por ello, el número de nodos explorados puede ser mucho mayor que la longitud del camino solución.

9. Resultados de la instrumentación

Se añadieron contadores para diferenciar los nodos realmente expandidos de los sucesores examinados.

Cuadro 3: Resultados obtenidos

Concepto	Primero el Mejor	A*
Nodos expandidos	548	283
Sucesores examinados	1499	747
Ramificación promedio	2.735	2.640

En Primero el Mejor, de los 1499 sucesores examinados, 563 ya habían sido visitados y fueron descartados. Por tanto, este contador representa el número total de sucesores evaluados y no el número de nodos únicos.

La distribución de la ramificación fue:

Cuadro 4: Distribución de la ramificación

Posición	Sucesores	Cantidad
Esquina	2	209
Borde	3	271
Centro	4	67

Los valores suman 547 en lugar de 548 debido a una diferencia de una unidad en la forma en que se registran los contadores durante la ejecución.

Finalmente, $548 \times 2.735 \approx 1499$, lo que confirma que los 1499 corresponden aproximadamente a los sucesores examinados y no a 1499 nodos únicos.

A* expande menos nodos, 283 frente a 548, porque utiliza la función $f(n) = g(n) + h(n)$, que permite priorizar los estados más prometedores.

10. Por qué los resultados son tan distintos

Toda la diferencia observada puede reducirse a una única decisión de diseño: la información utilizada por cada algoritmo para ordenar la cola de prioridad.



10.1. Primero el Mejor

Primero el Mejor ordena los nodos utilizando únicamente:

$$f(n) = h(n).$$

Por lo tanto, persigue el estado que parece estar más cerca de la meta, ignorando el costo acumulado para llegar hasta él.

Esto puede provocar que el algoritmo avance por caminos largos si, en cada momento, el valor heurístico parece favorable.

10.2. A*

A* utiliza:

$$f(n) = g(n) + h(n).$$

De esta forma, el algoritmo considera tanto la distancia ya recorrida como la distancia estimada restante.

El término $g(n)$ penaliza los caminos que ya han acumulado un costo elevado, incluso cuando presentan un buen valor heurístico.

Esta diferencia explica directamente los resultados obtenidos:

- Primero el Mejor encontró una solución de 62 movimientos.
- A* encontró una solución de 20 movimientos.
- Primero el Mejor expandió 548 nodos únicos.
- A* expandió 283 nodos únicos.

Por tanto, A* encontró una solución considerablemente más corta y necesitó expandir menos nodos.

11. ¿Qué método conviene usar?

Para el problema del 8-Puzzle, A* constituye la alternativa más adecuada cuando se busca una solución óptima.

En el experimento realizado, A* expandió menos de la mitad de los nodos realmente expandidos por Primero el Mejor:



$$\frac{283}{548} \approx 0.516.$$

Además, encontró una solución de 20 movimientos, mientras que Primero el Mejor encontró una de 62 movimientos.

La optimalidad de A^* está respaldada por el uso de la distancia Manhattan como heurística admisible.

Por otro lado, no se observó un costo significativo en tiempo de ejecución para una instancia de este tamaño:

$$t_{\text{Primero el Mejor}} \approx 0.0165 \text{ s},$$

$$t_{A^*} \approx 0.0166 \text{ s}.$$

Primero el Mejor podría ser preferible si el objetivo fuera obtener rápidamente cualquier solución sin importar su longitud. Sin embargo, para el 8-Puzzle, donde se dispone de una heurística informativa como la distancia Manhattan, A^* ofrece una ventaja clara al garantizar la optimalidad de la solución.

12. Conclusiones

El análisis experimental demuestra que la principal diferencia entre Primero el Mejor y A^* se encuentra en la función utilizada para seleccionar los nodos.

Primero el Mejor utiliza únicamente la información heurística:

$$f(n) = h(n),$$

mientras que A^* considera tanto el costo acumulado como la estimación del costo restante:

$$f(n) = g(n) + h(n).$$

Esta diferencia tiene consecuencias importantes. En la instancia analizada, Primero el Mejor obtuvo una solución de 62 movimientos y expandió 548 nodos únicos, mientras que A^* obtuvo una solución de 20 movimientos y expandió solamente 283 nodos únicos.

Asimismo, el análisis del contador de sucesores demuestra que el número de “nodos expandidos” reportado originalmente por el código no corresponde directamente al número de nodos únicos procesados. En Primero el Mejor se examinaron 1499 sucesores, pero únicamente 548 nodos fueron realmente expandidos.



Finalmente, los resultados confirman que A^* es la opción más apropiada para este problema cuando se requiere una solución de costo mínimo. La distancia Manhattan proporciona una heurística admisible que permite a A^* garantizar la optimalidad, mientras que Primero el Mejor sacrifica dicha garantía a cambio de una estrategia de búsqueda basada exclusivamente en la estimación heurística.

13. Anexos

Se anexan los archivos correspondientes a la implementación de los algoritmos **Primero el Mejor** (*Greedy Best-First Search*) y **A^*** , junto con las estructuras y funciones auxiliares utilizadas en el desarrollo del 8-Puzzle.

- `primero_mejor.py`: implementación de Primero el Mejor.
- `a_estrella.py`: implementación de A^* .
- `estado.py`: representación de los estados.
- `nodo.py`: estructura de los nodos.
- `heuristica.py`: implementación de la distancia Manhattan.
- `lista_prioridad.py`: implementación de la cola de prioridad.
- `problema.py`: definición del problema.
- `comprobar_solubilidad.py`: comprobación de solubilidad.
- `main.py`: ejecución principal del programa.